

Detailed Design and Project Timeline

Distributed Garbage Collection System

Course: ECE 498A

Group ID: 2015.034

Group Member Name	UW Id/user#

(names removed)	

Date of Submission: Thursday, July 3, 2014

Strengths of this report include the excellent justification and analysis, as well as use of upper-year knowledge, in Section 3.1

Table of Contents

1. High Level Description of Project	3
1.1 Motivation.....	3
1.2 Project Objective.....	3
1.3 Block Diagram	4
2. Project Specifications	5
2.1 Functional Specifications	5
2.2 Non-Functional Specifications	6
3. Detailed Design	6
3.1 Garbage Bin Allocation Subsystem	6
3.1.1 Problem Characterization	6
3.1.2 Problem Formulation	7
3.1.3 Problem Modelling and Design Alternatives	8
3.2 Network Communications Subsystem	17
3.2.2 Design Iterations Followed	19
3.2.3 Design Decisions and Alternatives Considered	20
3.2.4 Analysis	21
3.3 Load Sensor Subsystem.....	24
3.3.1 Design Iterations	26
3.3.2 Alternatives Considered.....	27
3.3.3 Analysis	29
3.4 Battery Sensor Subsystem	29
3.4.1 Detailed Description	29
3.4.2 Project Specifications Satisfied	30
3.4.3 Design Alternatives Considered	31
4.1 Discussion on Design.....	31
4.1.1 Scheduling to Date	31
4.1.2 Upper Year Knowledge Incorporated	32
4.1.3 Critical Analysis	32
4.1.4 Potential Hazards	33
4.2 Project Timeline	33
References	36

1. High Level Description of Project

1.1 Motivation

The allocation of garbage receptacles in outdoor settings at times appear to be inefficient or arbitrary, with many bins having been placed on the assumption of where they are needed most. Additionally, to service these receptacles requires maintenance and park staff be on a common reoccurring schedule and route, often regardless of whether or not these bins are filled to capacity. Because of this common scheduling, it can often be seen when attending a closed outdoor venue that garbage bins are often either left overflowing for some time before they are serviced or are emptied prematurely. This in turn means either loss of resources or productivity, or results in the act of littering by park guests and campus visitors. In fact, a number of official reasons can be attributed specifically to the existence and use of garbage bins [1] [2]. Whether bins are in the wrong place at the wrong time, not emptied out often enough or there are simply not enough bins at that instance, the Distributed Garbage Collection System offers added convenience to users while actively providing a means to prevent the generation of litter so individuals may never concern themselves with what to do with their trash.

1.2 Project Objective

Given the problem of litter and poorly allocated garbage collection resources, the Distributed Garbage Collection System sets out to provide a solution by means of hardware and software. Specifically, the objective holds that by providing a number of garbage receptacles, communicating as a network, can result in reduced garbage and litter and aid in efficient resource allocation of garbage collection operations. These robotic garbage bins will ultimately be able to detect capacity, whether this metric is met by volume or mass. At this point these bins will be able to notify personnel of their complete capacity and autonomously navigate themselves to a service station where they can be emptied and redeployed automatically based on demand in specific locations.

These bins essentially allow for the elimination of inefficiencies found in human controlled garbage collection protocols in closed outdoor settings, such as theme parks. Deployment is determined using knowledge of past usage statistics and replacement is done automatically without aid of human intervention solving two of the obvious problems (the where and when) that come to mind when one thinks of garbage collection. Although alternatives may exist for the problem of garbage collection (perhaps routed garbage chutes), this Distributed Garbage Collection System offers a simpler, more cost effective answer using engineering concepts including artificial intelligence, distributed computing, computer networks and statistics in addition to other engineering hardware design principles.

1.3 Block Diagram

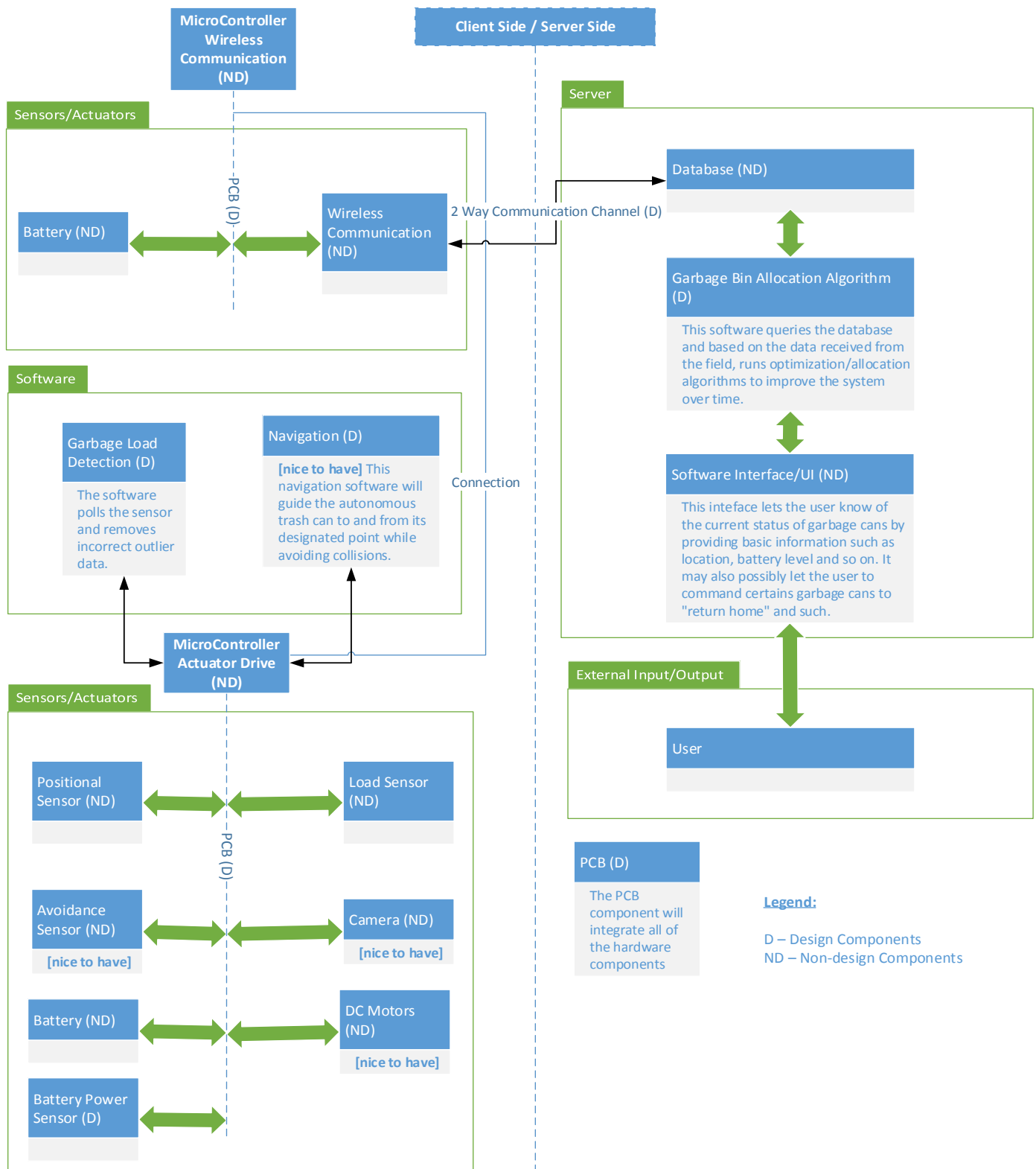


Figure 1 – Design Block Diagram

2. Project Specifications

2.1 Functional Specifications

Functional Specification	Essential (Yes/No)
There must be a latency of under 5 seconds when measuring garbage load.	Yes
The garbage bin must broadcast its status at least once every minute.	Yes
The embedded software must be able to measure and report garbage load measurements that are within $\pm 10\%$ accuracy of a manually conducted garbage load measurement.	Yes
A visible indicator on the garbage bin (e.g., LED light) should go on in the event that the garbage bin disconnects from the wireless network. Likewise, the receiver should be aware that a given garbage bin has disconnected from the wireless network. Both should occur within a time frame of 5 minutes after the garbage bin has disconnected from the network.	Yes
Sensor readings that differ by $\pm 40\%$ from previous and next readings should be discarded as outlier values.	Yes
A user interface on the server should display the status of each garbage bin currently deployed into the field. The status of each garbage bin should be accurate as of the past minute.	Yes
The system must be able to infer the frequency of usage from received data.	Yes
The garbage bin must broadcast a request for garbage collection when it is within 90% to 100% filled to capacity.	Yes
If a garbage bin is unable to contact the server for a specified period of time, it will enter into an idle state and wait for a response from the server.	Yes
A garbage bin must be able to communicate its location coordinates to the server.	Yes
The cost to be minimized by the garbage bin allocation algorithm must account for the maximum traversal time, the remaining garbage capacity and the remaining power of each garbage bin.	Yes
The runtime of the garbage bin allocation algorithm must not exceed 10 minutes.	Yes
The garbage bin allocation algorithm must always yield a semantically valid allocation. An allocation is defined to be semantically valid if and only if the allocation contains at least one garbage bin, each patrol position is assigned to one and only one garbage bin, and the path created by linking the assigned patrol positions has not nested cycles and terminates at a service station.	Yes
The garbage bin allocation algorithm is best-effort and does not guarantee convergence to an allocation that yields the lowest minimum cost.	Yes
The garbage bin allocation algorithm must be able to reallocate garbage bins already in the field.	Yes

If the server does not receive a ping from a given garbage bin within a maximum interval of 10 minutes, then that garbage bin is considered dead and must be serviced. In this situation, the server should allocate a garbage bin in its place.	No
The garbage bin should start traversing to the nearest servicing station within 2 minutes of receiving a request from the server.	No
The garbage bin should be able to avoid obstacles while moving. Specifically, the proximity between the moving garbage bin and a non-moving obstacle should be no closer than 1 cm.	No
The garbage bin must be able to measure its remaining battery level. The accuracy of these battery level measurements should be within $\pm 10\%$ of the measurements conducted using a commercial tester.	No
The garbage bin should be able to broadcast a request for service if its battery level runs below 20%.	No
Upon change from idle state, the garbage bin will engage in a security protocol exchange with the initiator to verify if the initiator is an authentic server.	No

2.2 Non-Functional Specifications

Functional Specification	Essential (Yes/No)
The system should last at least 6 hours on a single battery charge.	Yes
The sensors and the associated circuitry must not obstruct normal operation. The incoming trash must not touch the sensor. It must have at least 1 cm of clearance to ensure no contact.	Yes
The system should be scalable and support at least 70 garbage bins. This can be verified via simulation.	No
The circuitry for sensing and wireless logic should fit in a 20 cm by 20 cm by 15 cm enclosure.	No
A user should not need to spend more than 10 seconds to verify the status of a garbage bin.	No

3. Detailed Design

3.1 Garbage Bin Allocation Subsystem

3.1.1 Problem Characterization

One of the subsystems to be developed in this project is a garbage bin allocation algorithm for allocating patrol positions to garbage bins in an open space. To develop this subsystem, the first design iteration will involve characterizing the optimization problem to be solved relative to the following use case. In this use case, it is assumed that the system has been running for a long time. In the current time unit, several garbage bins in the field have indicated that they require servicing. Hence, the server must now determine

how it is going to allocate these vacated patrol positions to the remaining available garbage bins. To complicate matters, the set of available garbage bins and the set of available patrol positions are not necessarily homogeneous. In particular, each garbage bin can be differentiated by location, travelling speed, remaining garbage capacity and remaining power level. Likewise, patrol positions can be differentiated by location and average production of garbage within any given hour.

Given the heterogeneity between available garbage bins and between available patrol positions, the allocation problem presented here is no longer as clear cut as simply placing an available garbage bin at an available patrol position. In particular, the garbage bin allocation algorithm must somehow quantify the cost associated with any given allocation of available patrol positions to available garbage bins. In particular, such a multi-objective cost function should attempt to minimize the traversal time of each available garbage bin to their allocated patrol positions so as to minimize the overhead of conventional garbage collection. Additionally, penalties should be imposed if an available garbage bin is assigned to more patrol positions than it can handle in terms of capacity. As well, garbage bins with low remaining power should not be allocated too many patrol positions in order to minimize the time overhead associated with collecting a dead garbage bin in the field.

3.1.2 Problem Formulation

Based on the problem characterization provided in Section 3.1.1, the optimization problem for garbage bin allocation can be formulated as follows:

Let $R = \{r_1 r_2 r_3 \dots r_n\}$ be the set of available garbage bins.

Let $T = \{t_1 t_2 t_3 \dots t_m\}$ be the set of available patrol positions.

Let $D = \{d_1 d_2 d_3 \dots d_k\}$ be the set of registered servicing stations.

Let $\mathbf{c} = (c_1 c_2 c_3 \dots c_n)$ be a vector of available garbage capacities, where c_i is the available garbage capacity of r_i for $1 \leq i \leq n$.

Let $\mathbf{v} = (v_1 v_2 v_3 \dots v_n)$ be a vector of garbage bin speeds, where v_i is the speed of r_i for $1 \leq i \leq n$.

Let $\mathbf{p} = (p_1 p_2 p_3 \dots p_n)$ be a vector of remaining power percentages, where $0 \leq p_i \leq 1$ is the percentage of power remaining in r_i for $1 \leq i \leq n$.

Let $\lambda = (\lambda_1 \lambda_2 \lambda_3 \dots \lambda_n)$ be a vector of mean garbage productions in the current hour, where λ_i is the total mean amount of garbage produced by all patrol positions allocated to r_i for $1 \leq i \leq n$.

Given the parameters defined above, the objective of the optimization problem is to find an allocation $S = \{ (r_1 T_1 D_1) (r_2 T_2 D_2) \dots (r_n T_n D_n) \}$ that minimizes the following cost function

$$f(S) = \sqrt{(total\ traversal\ time(S))^2 + (overload\ penalty(S))^2 + (low\ power\ penalty(S))^2} \quad (1)$$

$$total\ traversal\ time(S) = \sum_{i=1}^n \frac{length(shortest\ path(r_i, T_i D_i))}{v_i} \quad (2)$$

$$\text{overload penalty}(S) = 100 \sum_{i=1}^n \Pr(X > c_i) = 100 \sum_{i=1}^n (1 - \Pr(X \leq c_i)) \quad (3)$$

$$\Pr(X \leq c_i) = e^{-\lambda_i} \sum_{j=1}^{\lfloor c_i \rfloor} \frac{\lambda_i^j}{j!} \quad (4)$$

$$\text{low power penalty}(S) = 100 \sum_{i=1}^n |T_i|(1 - p_i) \quad (5)$$

subject to

$$T_i \subseteq T, 1 \leq i \leq n \quad (6)$$

$$T_i \cap T_j = \emptyset, 1 \leq i \neq j \leq n \quad (7)$$

$$D_i \subseteq D, 1 \leq i \leq n \quad (8)$$

$$|D_i| = 1, 1 \leq i \leq n \quad (9)$$

In terms of formulas, formula (1) defines the overall cost function to be minimized by the garbage bin allocation subsystem. Here, formula (1) consists of three components: the total traversal time, the overload penalty, and the low power penalty. The total traversal time component is computed using formula (2) and essentially computes the sum of traversal times over all available garbage bins for a given garbage bin allocation. Meanwhile, the overload penalty component computed using formula (3) effectively serves as an additional cost that is meant to penalize garbage bin allocations that have a high probability of overfilling a subset of the garbage bins in the allocation. In particular, each garbage bin r_i has a remaining garbage capacity c_i that should preferably not be exceeded. The probability that this remaining capacity c_i is not exceeded that is accounted for in (3) is modelled using the cumulative Poisson distribution formula defined by (4). Finally, the low power penalty component is computed using formula (5) and as with overload penalty, is meant to penalize allocations that allocate too many patrol positions to garbage bins running low on power.

In terms of constraints, the meaning behind constraints (6) and (7) is that T_i represents the subset of available patrol positions T that is allocated to r_i , where each available patrol position can only be assigned to a single garbage bin r_i for $1 \leq i \leq n$. Meanwhile, (8) and (9) dictate that D_i is the registered servicing station that the path assigned to garbage bin r_i is expected to terminate at for $1 \leq i \leq n$.

3.1.3 Problem Modelling and Design Alternatives

3.1.3.1 Decomposition to Multi-Robot Task Allocation (MRTA) Problem

Now that the optimization problem has been characterized and formulated, the next step in terms of design iteration is to find an appropriate model for solving it. In particular, one of the concepts learned in ECE 457A Cooperative and Adaptive Algorithms is that an optimization problem in practice can be modelled after a theoretical optimization problem. In this case, it turns out that this garbage bin allocation problem resembles the Multi-Robot Task Allocation problem that is discussed in [3] and [4]. To see why this is the case, consider the definition for Multi-Robot Task Allocation from [3] and [4]:

Let $R = \{R_1, R_2, \dots, R_n\}$ be the set of available heterogeneous robots [3] [4].

Let $T = \{T_1, T_2, \dots, T_m\}$ be the set of tasks to be allocated to each available heterogeneous robot [3] [4].

Let $A : T \rightarrow R$ be the allocation function that allocates the tasks in T to the available heterogeneous robots in R [3] [4].

Then the objective is to find an optimal allocation $S = \{ (R_1 T_1) (R_2 T_2) \dots (R_k T_k) \}$ that minimizes some cost function $f(S)$ [3] [4].

Based on the definition for the Multi-Robot Task Allocation problem provided by [3] and [4], it is evident that there are very clear parallels between the garbage bin allocation problem and the Multi-Robot Task Allocation problem. In particular, the available garbage bins can be likened to the available heterogeneous robots discussed in [3] and [4]. Meanwhile, the patrol positions and registered servicing stations to be allocated to each available garbage bin can be analogized to the tasks to be assigned to each function. Hence, the models used to solve the Multi-Robot Task Allocation problem should also be applicable to the garbage bin allocation problem. From this point forward, the critical design decision to be made is the model that should be adopted for solving the garbage bin allocation problem.

3.1.3.2 mTSP-Based MRTA for Garbage Bin Allocation

One of the models that is predominantly used for solving the Multi-Robot Task Allocation problem is the Multiple Travelling Salesman problem (mTSP) [3] [4]. Hence, the objective of this subsection will be to develop a solution model for the garbage bin allocation problem based on mTSP. In general, mTSP is considered a generalization of the Travelling Salesman problem (TSP). In TSP, there exists a salesman who must visit m cities exactly once [5]. Given these m cities and the distances between any pair of these cities, the optimization objective is to find the shortest route without nested cycles that connects all m cities and returns to the first city visited [5]. In mTSP, however, the optimization objective in TSP is relaxed to include n salesmen [5]. Hence, the optimization objective for mTSP is to find an allocation of cities to salesmen with the hard constraint that each city can only be allocated to one salesman and visited once such that the total travelling distance of all n salesman is minimized [5]. In this sense, the subset of cities to be visited by a given salesman can be considered its own TSP [5].

Based on the provided definition of mTSP, the garbage bin allocation problem can be modelled as follows. In particular, the garbage bins as robots can further be analogized as salesmen, while the patrol positions and servicing stations as tasks can be further analogized as the cities to be visited. Additionally, since the garbage bins can start and end at different servicing stations, the hard constraint in mTSP requiring that each route return to its starting city can be eliminated. Hence, this results in the mTSP solution model shown in Figure 1 and Figure 2 as adapted from [3].



Figure 2: The mTSP Solution Model for Garbage Bin Allocation with 3 Garbage Bins and 9 Patrol Positions



Figure 3: Terminating Service Station Permutation for 3 Garbage Bins

Figure 1 describes the allocation of available patrol positions to the available garbage bins (e.g., patrol positions t_1 , t_2 , and t_3 are allocated to garbage bin r_1). Figure 1 also defines the order in which each available patrol position is visited (e.g., garbage bin r_2 will visit patrol positions t_4 , t_5 , t_6 and t_7 in order). Meanwhile, Figure 2 describes the service stations that each garbage bin will terminate at (e.g., garbage bin r_3 will traverse to and terminate at service station d_3 after visiting patrol position t_9).

The constraints to the mTSP solution model shown in Figure 1 and 2 should also be stated. In particular, a valid garbage bin and patrol position permutation such as the one shown in Figure 1 must always start on the left with a garbage bin. This is because the specifications dictate that all patrol positions must be assigned to a garbage bin. Hence, if a garbage bin and patrol position permutation starts on the left with a patrol position, this implies that there are patrol positions not assigned to a garbage bin. Furthermore, the number of elements in a valid garbage bin and patrol position permutation must be equal to the sum of the cardinalities of the set of available garbage bins and the set of available patrol positions. Finally, a valid garbage bin patrol position permutation must not contain any repeated elements. Meanwhile, the number of elements in a valid terminating service station permutation must be equal to the cardinality of the set of available garbage bins. Unlike the garbage bin and patrol position permutation, however, a valid terminating service station permutation can contain repeated elements. This is to illustrate the fact that multiple garbage bins can terminate at the same service station.

Given the mTSP solution model and its constraints, it is now possible to solve for a best-effort allocation S that minimizes the cost function defined in formula (1) of Section 3.1.2. ECE457A discusses numerous general search algorithms that can be applied towards solving an optimization problem such as garbage bin allocation. For the purposes of this design project, however, the simulated annealing-based Multi-Robot Task Allocation algorithm proposed by [3] should suffice (see Algorithm 2 in [3]). For completeness, a slightly modified version of this algorithm from [3] adapted for the context of garbage bin allocation is provided below as Algorithm 1.

Algorithm 1: Pseudo-Code for mTSP-Based Garbage Bin Allocation Using Simulated Annealing [3]

Garbage Bin Allocation-Specific Input Parameters:

- 1 garbageBins: The list of available garbage bins.
- 2 patrolPositions: The list of available patrol positions.
- 3 serviceStations: The list of registered servicing stations.

Simulated Annealing Input Parameters:

- 4 initialTemp: The initial temperature value for Simulated Annealing.
- 5 finalTemp: The final temperature value for Simulated Annealing.
- 6 iterationsPerTemp: Number of iterations per temperature value for Simulated Annealing.
- 7 maxIterations: The maximum number of iterations to run Simulated Annealing for.

Output Parameters:

- 8 optimalBinAlloc: The lowest cost allocation of patrol positions to available bins.
- 9 optimalStationAlloc: The lowest cost terminating service station assignment for each bin.
- 10 optimalCost: The cost associated with optimalBinAlloc and optimalStationAlloc.

Variable Definitions:

- 11 currentTemp: The current temperature value for Simulated Annealing.
- 12 coolingFactor: The cooling factor for geometric cooling under Simulated Annealing.
- 13 transitionProb: The transition probability for a neighbor allocation of higher cost.
- 14 currentBinAlloc: The current allocation of patrol positions to available bins.

15 neighborBinAlloc: The neighboring allocation of patrol positions to available bins.
 16 currentStationAlloc: The current terminating service station assignment for each bin.
 17 neighborStationAlloc: The neighboring terminating service station assignment for each bin.
 18 currentCost: The cost associated with currentBinAlloc and currentStationAlloc.
 19 neighborCost: The cost associated with neighborBinAlloc and neighborStationAlloc.
 20 randomNumber: Stores a randomly generated number n where $0 \leq n \leq 1$.
 21 iterations: The current iteration in the algorithm.

Algorithm:

```

22 currentBinAlloc ← generateValidPermutation( garbageBins, patrolPositions )
23 currentStationAlloc ← findNearestServiceStations( currentBinAlloc )
24 optimalCost ← computeCost( currentBinAlloc, currentStationAlloc )
25 currentTemp ← initialTemp
26 iterations ← 1
27 while currentTemp > finalTemp and iterations ≤ maxIterations do
28   for i ← 1 to iterationsPerTemp do
29     neighborBinAlloc ← generateNeighborBinAlloc( currentBinAlloc )
30     neighborStationAlloc ← findNearestStations( neighborBinAlloc, serviceStations )
31     neighborCost ← computeCost( neighborBinAlloc, neighborStationAlloc )
32     if neighborCost < currentCost then
33       currentBinAlloc ← neighborBinAlloc
34       currentStationAlloc ← neighborStationAlloc
35       currentCost ← neighborCost
36       if neighborCost < optimalCost then
37         optimalBinAlloc ← neighborBinAlloc
38         optimalStationAlloc ← neighborStationAlloc
39         optimalCost ← neighborCost
40       end
41     else
42       randomNumber ← generateRandomNumber( 0, 1 )
43       transitionProb ← exp( -1 × ( ( neighborCost - currentCost ) / currentTemp ) )
44       if transitionProb > randomNumber then
45         currentBinAlloc ← neighborBinAlloc
46         currentStationAlloc ← neighborStationAlloc
47         currentCost ← neighborCost
48       end
49     end
50   end
51   currentTemp ← currentTemp × coolingFactor
52   iterations ← iterations + 1
53 end
  
```

The simulated annealing-based approach used by Algorithm 1 to arrive at a best-effort garbage bin allocation S works as follows. In the simulated annealing search algorithm, the termination condition is defined using the concept of a cooling schedule. As part of this cooling schedule, the algorithm will track its current temperature (defined in line 11 of of Algorithm 1). Initially, the current temperature will be set to some initial temperature (defined in line 4 of Algorithm 1) such as 100°C. Every few iterations of the algorithm (the number of iterations defined in line 6 of Algorithm 1), the current temperature will drop by a cooling factor (see line 51 of Algorithm 1). When the current temperature drops below a certain threshold temperature (defined in line 5 of Algorithm 1) such as 1°C, the algorithm terminates. Additionally, since the number of iterations is to be bounded, Algorithm 1 will also terminate if iterations (defined in line 21 of Algorithm 1) exceeds maxIterations (defined in line 7 of Algorithm 1).

Nested within the cooling schedule for simulated annealing is the optimization algorithm for mTSP garbage bin allocation. Before executing the cooling schedule, Algorithm 1 initializes the current garbage

bin allocation to a valid random garbage bin allocation S for the given set of garbage bin allocation inputs. Specifically, `currentBinAlloc` (defined in line 14 of Algorithm 1) and `optimalBinAlloc` (defined in line 8 of Algorithm 1) are initialized to a valid garbage bin and patrol position permutation resembling Figure 1, while `currentStationAlloc` (defined in line 16 of Algorithm 1) and `optimalStationAlloc` (defined in line 9 of Algorithm 1) are initialized to a valid service station permutation resembling Figure 2 (see lines 22 to 25 in Algorithm 1). While executing the cooling schedule, however, Algorithm 1 will perturb the current garbage bin allocation to obtain a neighboring garbage bin allocation. This is accomplished by randomly performing one of the operations defined in Figure 3 (taken from [3]) on `currentBinAlloc` in order to obtain `neighborBinAlloc` (defined in line 15 of Algorithm 1) and deterministically computing `neighborStationAlloc` (defined in line 17 of Algorithm 1) accordingly. If it turns out that the cost of neighboring garbage bin allocation is less than the cost of the current garbage bin allocation, Algorithm 1 will assign the neighboring garbage bin allocation as the current garbage bin allocation (see lines 32 to 35 in Algorithm 1). If it turns out further that the neighboring garbage bin allocation is less costly than the current optimal garbage bin allocation, then Algorithm 1 will assign the neighboring garbage bin allocation as the optimal garbage bin allocation (see lines 36 to 39 in Algorithm 1). Otherwise, if it turns out that the neighboring garbage bin allocation is more costly than the current optimal garbage bin allocation, Algorithm 1 will probabilistically select the neighboring garbage bin allocation as the current garbage bin allocation (see lines 42 to 47 in Algorithm 1). In particular, Algorithm 1 will compute a transition probability and generate a random real number between 0 inclusive and 1 inclusive. If the transition probability is greater than this random number, then Algorithm 1 will assign the more costly garbage bin allocation as the current garbage bin allocation. The purpose of intentionally assigning a more costly neighboring garbage bin allocation as the current garbage bin allocation is to prevent Algorithm 1 from getting stuck in a local optima. When the cooling schedule terminates, `optimalBinAlloc` and `optimalStationAlloc` are returned.

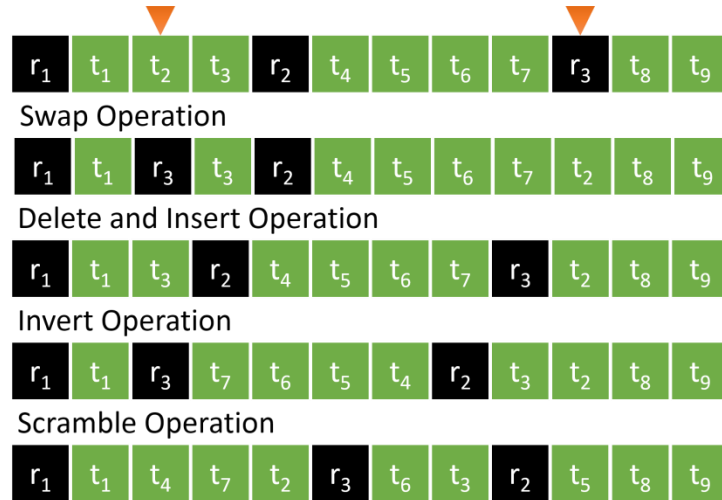


Figure 4: Operations for Generating Neighboring Garbage Bin and Patrol Position Permutations [3]

3.1.3.3 Simple Auction-Based MRTA for Garbage Bin Allocation

Another model discussed in [4] that is occasionally used for solving the Multi-Robot Task Allocation problem is the Simple Auction problem. The purpose of this subsection will be to develop a Simple Auction-based solution model as a possible alternative for solving the garbage bin allocation problem.

In the Simple Auction problem, there exists an auctioneer who is auctioning away a single item and a set of n bidders, each of whom desire to possess this item. Each of the n bidders will submit a bid and the item is sold to the highest bidder. Based on this protocol, the server can be analogized as the auctioneer, the available garbage bins as robots can be analogized as the bidders and the patrol positions as tasks can be analogized as items to be auctioned off. Hence, a possible solution model for garbage bin allocation based on the Simple Auction problem is represented by Figures 2 and 4.

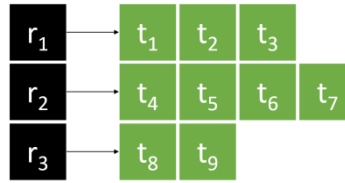


Figure 5: A Simple Auction Solution Model for Garbage Bin Allocation with 3 Bins and 9 Patrol Positions

As shown in Figure 4 above, the allocation of patrol positions to garbage bins can be represented as an array of lists, where each index i into the array corresponds to a garbage bin r_i in the set of available garbage bins. In this case, garbage bin r_1 is allocated patrol positions t_1 , t_2 and t_3 ; garbage bin r_2 is allocated patrol positions t_4 , t_5 , t_6 and t_7 ; and garbage bin r_3 is allocated patrol positions t_8 and t_9 . It should be noted that the order of each patrol position in the list dictates the order in which they are visited. For example, garbage bin r_1 will visit t_1 first, t_2 second, and t_3 third. It should also be noted that patrol positions can only be allocated to a single garbage bin. As for the assignment of terminating service stations to garbage bins, the same rules and constraints from the mTSP solution model apply. Given this solution model and its constraints, the following algorithm (i.e., Algorithm 2) partially adapted from [4] can be used to solve for a best-effort garbage bin allocation with respect to cost function (1).

Algorithm 2: Pseudo-Code for Simple Auction-Based Garbage Bin Allocation [4]

Input Parameters:

- 1 garbageBins: The list of available garbage bins.
- 2 patrolPositions: The list of available patrol positions.
- 3 serviceStations: The list of available service stations.
- 4 initialBinAlloc: The initial allocation of patrol positions to available bins.
- 5 initialStationAlloc: The initial assignment of service stations to available bins.

Output Parameters:

- 6 optimalBinAlloc: The lowest cost allocation of patrol positions to available bins.
- 7 optimalStationAlloc: The lowest cost terminating service station assignment for each bin.

Variable Definitions:

- 8 currentBinAlloc: The current allocation of patrol positions to available bins.
- 9 currentStationAlloc: The current terminating service station assignment for each bin.
- 10 lowestBid: The current lowest bid.
- 11 lowestBidder: The current lowest bidding garbage bin.

Algorithm:

```

12 currentBinAlloc ← initialBinAlloc
13 currentStationAlloc ← initialStationAlloc
14 foreach patrolPosition in patrolPositions do
15     lowestBid ← ∞
16     foreach garbageBin in garbageBins do
17         currentBid ← getBid( garbageBin, currentBinAlloc, currentStationAlloc )
18         if currentBid < lowestBid then
19             lowestBid ← currentBid
20             lowestBidder ← garbageBin
21         end
22     end
23 currentBinAlloc ← append( patrolPosition, lowestBidder, currentBinAlloc )
24 currentStationAlloc ← findNearestStations( currentBinAlloc, serviceStations )
25 end
26 optimalBinAlloc ← currentBinAlloc
27 foreach garbageBin in garbageBins do
28     optimalBinAlloc ← optimizeTSP( garbageBin, optimalBinAlloc )
29 end
30 optimalStationAlloc ← findNearestStations( optimalBinAlloc, serviceStations )

```

The Simple Auction-based approach used by Algorithm 2 to arrive at a best-effort garbage bin allocation S with respect to cost function (1) for the given set of inputs (defined in lines 1 to 5 of Algorithm 2) works as follows. At the beginning of Algorithm 2, the current garbage bin allocation is initialized to the initial garbage bin allocation (see lines 12 to 13 in Algorithm 2). This represents the fact that each garbage bin may already be allocated to patrol positions in the field. Then for each available patrol position, an auction is held by Algorithm 2 (see lines 14 to 24 in Algorithm 2). The current lowest bidder with respect to the current patrol position being auctioned is allocated the patrol position. Once the auction is over, Algorithm 2 will then proceed to optimize the resulting garbage bin allocation (see lines 26 to 30 in Algorithm 2). In particular, the ordering of the patrol positions for each garbage bin is optimized as a Travelling Salesman Problem (TSP) using a search algorithm such as Simulated Annealing in order to minimize the distance traversed by each available garbage bin [4].

Given the structure of the Simple Auction problem, it is not possible to directly apply cost function (1) when computing the bids for each available garbage bin. Hence, the following utility function will be used to compute the bid u_i for each available garbage bin r_i instead:

$$u_i(t) = \sqrt{\left(\frac{\text{distance}(r_i, t)}{v_i}\right)^2 + (100 \times (1 - \Pr(X \leq c_i)))^2 + (100 \times (|T_i| + 1))^2} \quad (9)$$

where t is the patrol position being auctioned off, $\frac{\text{distance}(r_i, t)}{v_i}$ is the straight traversal time between the garbage bin's current position and the patrol position t , $\Pr(X \leq c_i)$ is the probability (see formula (4)) that the total garbage produced by all allocated garbage bins and t in the current hour is less than the remaining garbage capacity in garbage bin r_i , and $|T_i| + 1$ is the number of patrol positions currently assigned to r_i plus one.

3.1.4 Analysis

3.1.4.1 Overview

Given that the design alternatives for garbage bin allocation have now been defined, the next step in the design iteration will be to select the MRTA-based problem model that should be used to develop the garbage bin allocation algorithm. To meet this objective, both design alternatives will first be validated against the project specifications defined in Section 2. Afterwards, an engineering analysis will be carried out on both design alternatives to determine which one is more suitable for integration into this project.

3.1.4.2 Satisfaction of Project Specifications

For completeness, the project specifications from Section 2 that are relevant to the garbage bin allocation algorithm have been reproduced below:

1. The cost to be minimized by the garbage bin allocation algorithm must account for the maximum traversal time, the remaining garbage capacity and the remaining power of each garbage bin.
2. The runtime of the garbage bin allocation algorithm must not exceed 10 minutes.
3. The garbage bin allocation algorithm must always yield a semantically valid allocation. An allocation is defined to be semantically valid if and only if the allocation contains at least one garbage bin, each patrol position is assigned to one and only one garbage bin, and the path created by linking the assigned patrol positions has no nested cycles and terminates at a servicing station.
4. The garbage bin allocation algorithm is best-effort and does not guarantee convergence to an allocation that yields the lowest minimum cost.
5. The garbage bin allocation algorithm must be able to reallocate garbage bins already in the field.
6. The system should be scalable and support at least 70 garbage bins.

Given the design alternatives for garbage bin allocation defined in Section 3.1.3, it can be asserted that the problem specifications listed above are satisfied. With respect to specification 1, both the mTSP-based and the auction-based MRTA problem models utilize cost function (1), which accounts for the maximum traversal time, the remaining garbage capacity and the remaining power of each garbage bin. With respect to specification 3, the model-specific constraints defined in Sections 3.1.3.2 and 3.1.3.3 effectively enforce that the resulting allocations are semantically valid. With respect to specification 4, Algorithm 1 and Algorithm 2 both utilize simulated annealing for optimizing the allocation of garbage bins. In both cases, the runtime of simulated annealing is ultimately bounded by the parameters used to initialize its cooling schedule and not whether or not the lowest cost allocation has been found, hence best-effort. With respect to specification 5, both Algorithm 1 and Algorithm 2 permit allocation to be performed on garbage bins situated in the field and at a service station. Specifications 2 and 6 will be verified by simulation in the next subsection.

3.1.4.3 Simulation and Critical Evaluation

In order to ultimately select one of the design alternatives for integration into the garbage bin allocation subsystem, the two design alternatives considered in Section 3.1.3 will be systematically evaluated against the weighted decision matrix provided in Table 1 below.

Table 1: The Weighted Decision Matrix for Evaluating Garbage Bin Allocation Design Alternatives

Criteria	Weight	Points	Requirements
Cost	40%	See Req.	$Points = \begin{cases} \frac{2 \times 10^7 - Cost}{2 \times 10^7} \times 3, & cost < 2 \times 10^7 \\ 0, & otherwise \end{cases}$
Runtime	25%	See Req.	$Points = \begin{cases} \frac{600000\ ms - runtime}{600000\ ms} \times 3, & runtime < 600000\ ms \\ 0, & otherwise \end{cases}$
Time Complexity	10%	0	The algorithm tries to solve a problem that has undecidable time complexity (i.e., the algorithm might never terminate).
		1	The algorithm tries to solve a problem that has non-deterministic polynomial time complexity (i.e., the worst cast execution time is an exponential function of the number of patrol positions m).
		2	The algorithm tries to solve a problem that has polynomial time complexity (i.e., the worst case execution time is a polynomial function of the number of patrol positions m).
		3	The algorithm tries to solve a problem that has linear time complexity or better (i.e., the worst case execution time is at most a linear function of the number of patrol positions m).
Scalability	25%	0	Scalable to fewer than 10 garbage bins.
		1	Scalable to between 10 and 50 garbage bins.
		2	Scalable to between 51 and 99 garbage bins.
		3	Scalable to 100 or more garbage bins.

Table 1 will evaluate the design alternatives for garbage bin allocation against the following criteria: cost, runtime, time complexity, and scalability. Cost will evaluate the final cost of the garbage bin allocation resulting from the simulation of each algorithm for a fixed set of inputs. In general, an algorithm that yields a garbage bin allocation of higher cost will score lower in the cost criterion. Runtime will evaluate the time it takes for each algorithm to run from start to finish during simulation for a fixed set of inputs. Here, an algorithm that takes longer to run from start to finish will generally score lower for the runtime criterion. Time complexity will evaluate the time complexity of the problem models incorporated into each algorithm. Finally, the scalability criterion will evaluate the scalability of each algorithm in terms of the number of garbage bins that the algorithm can scale up to.

As shown in Table 1, the weightings for each of the criterion were assigned as follows. The cost criterion was assigned the highest weight of 40% because it is associated with the main objective to be fulfilled by the garbage bin allocation subsystem: to minimize the costs of garbage collection with respect to cost function (1). The runtime criterion and the scalability criterion were assigned the second highest weightings of 25% each because these criteria are considered the second most important with respect to the garbage bin allocation subsystem. In particular, an algorithm that is scalable to many garbage bins and patrol positions is desirable as it makes the algorithm more adaptive to different input sizes. Meanwhile, an algorithm that runs and completes garbage bin allocation in less than 10 minutes on average is

desirable so that the overall system has time to perform other useful jobs. Finally, the time complexity criterion was assigned a weight of 10% because it is considered the least important criterion in the list. In particular, while it is nice to have an algorithm that takes a greedy approach towards garbage bin allocation, cost minimization with respect to (1) is favored as a more important criterion.

To evaluate each design alternative under the cost, runtime, and scalability criteria, Algorithm 1 and Algorithm 2 were prototyped in Java 7 and simulated under a fixed set of randomly generated inputs common between both algorithms. The randomly generated inputs consisted of 4 statically defined service stations, 100 randomly generated garbage bins stationed at one of the 4 service stations, and 1000 randomly generated patrol positions in a 10000×10000 2D grid space, where distance was computed using the Euclidean distance formula. In this case, both algorithms were basically implemented from scratch, utilizing only the libraries provided in JDK 7 and the Apache Commons Math library for quick access to the Poisson distribution used in formula (1). The evaluation results for each algorithm against Table 1 are provided in Tables 2 and 3.

Table 2: Evaluation Results for Algorithm 1 (mTSP-based MRTA for Garbage Bin Allocation)

Criteria	Weight	Result	Points	Weighted Score
Cost	40%	$Cost = 9479340.32057687$	1.578098952	0.63123958
Runtime	25%	$Runtime = 330883 \text{ ms}$	1.345585	0.33639625
Time Complexity	10%	Requirement 2	1	0.1
Scalability	25%	Requirement 3	3	0.75
Total				1.81763583

Table 3: Evaluation Results for Algorithm 2 (Auction-based MRTA for Garbage Bin Allocation)

Criteria	Weight	Result	Points	Weighted Score
Cost	40%	$Cost = 1.6070560625093937 \times 10^7$	0.589415906	0.235766362
Runtime	25%	$Runtime = 119902 \text{ ms}$	2.40049	0.6001225
Time Complexity	10%	Requirement 2	1	0.1
Scalability	25%	Requirement 3	3	0.75
Total				1.685888862

Based on the results provided in Table 2 and Table 3, it can be ascertained that both alternatives satisfy specifications 2 and 6 in Section 3.1.4.2; however, since the highest weighted score was yielded by Algorithm 1, this implies that an mTSP-based MRTA problem model should be integrated into the garbage bin allocation subsystem.

3.2 Network Communications Subsystem

The server needs a means of communicating with the clients. Communication was one of the design components and encompasses various protocols. The clients in this system are the garbage cans located in the field and the server is a central computing system with a database and algorithms which manipulate it as shown in the Figure below.

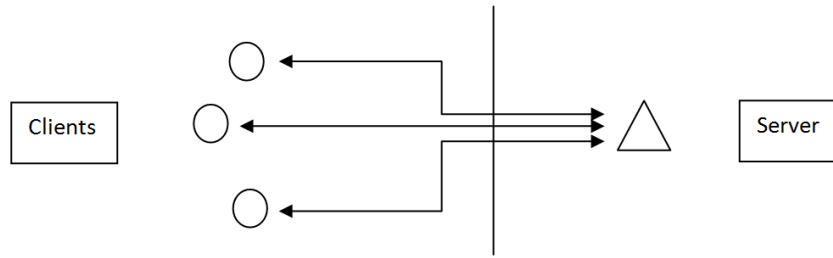


Figure 6: Client Server Model

The duplex communication allows the clients to post their status to the server and allows the server to apply an allocation algorithm that tells the clients where to go. The communication between server and clients is done through a UDP protocol. A client can send a specifically formed string to the server. A server then parses the string and fills out its database tables. This is largely the functionality of this subsystem. The database tables are then used by the allocation algorithm (a different subsystem). The string format needs to convey the information efficiently.

This system will be centralized, where the server contains much of the complexity which includes: communication protocol and the allocation algorithm. The server then commands the clients which are in the field by the means of wireless communication (Wi-Fi, 3G/4G).

Table 4: Specifications Satisfied by this Subsystem

Functional/Non-functional Spec	Essential?
A) The garbage bin must broadcast its status at least once every minute.	Yes
B) A UI on the server should display the status of each garbage bin currently deployed into the field. These statuses should be accurate as of the past minute.	Yes
C) If the server does not receive a ping from a given garbage bin within a maximum interval of 10 minutes, then that garbage bin is considered dead and must be serviced. In this situation, the server should allocate an available garbage bin in its place.	No
D) The system should be scalable and support at least 70 garbage cans. This can be verified via simulation.	No
E) A user should not need to spend more than 10 seconds to verify the status of a garbage can.	No
F) A garbage bin must be able to communicate its location coordinates to the server.	Yes
G) The system must be able to infer the frequency of usage from received data.	Yes

3.2.2 Design Iterations Followed

The design iteration protocol which the group decided to go with for this particular feature was: prototyping -> testing -> analyzing -> refining

The group found that the above approach was particularly useful for the software component because prototyping stage was doable given that the subsystem is software. This resulted in much more rapid convergence towards a working solution. Convergence was faster because errors were allowed to be made during the prototyping stage only at the cost of the development time. Additionally, having an initial iteration that worked aided the group in understanding the amount of time the subsystem would require until it was finished. This of course also helped in the project management aspect.

Design iterations listed below contain specific references to courses which were required in designing the subsystem during the prototyping stages.

Table 5: Server Design Iterations

Design Iteration	Actions
Prototype	Send a single ping to the newly created server (ECE 358)
Test	Ensure that the response was being received
Analyze	Simple case worked, need to make an elaborate case
Refine	Need to add a client server model
Prototype	Created a local java client program that sends a string, and created a local java server program. Created this client program with knowledge of ECE 454 assignment
Test	Ensure that the string is being bounced back through a loopback IP address
Analyze	An isolated case worked, need to move to an actual server
Refine	Need to read familiarize with Ubuntu server, move an implementation to Ubuntu
Prototype	Installed JDK on the Ubuntu server, compiled previously created files and ran (ECE 454)
Test	Sent a message from a remote desktop to the server, and received it.
Analyze	An ideal case worked, need to try a different client
Refine	Need to create a client in C
Prototype	Wrote a simple C client, that sent the same message string to the server
Test	Successfully received a string on a Java server

Table 6: Database Design Iterations

Design Iteration	Actions
Prototype	Run MySQL on windows (ECE 356)
Test	Try creating and inserting statements through UI
Analyze	Simple case worked, now try to do the same with code
Refine	Need to create a Java class with JDBC (Java Database Connectivity)

Prototype	Created a local java program which manipulates a table (ECE 356 + ECE 454)
Test	Ensure that entries are present through MySQL workbench interface
Analyze	Code worked on the local machine, needs to work now on the Ubuntu server
Refine	Need to read familiarize with Ubuntu server, move an implementation to Ubuntu
Prototype	Installed MySQL on the server, reconfigured it, tried inserting statements through the command-line interface (ECE 356 + ECE 454)
Test	Ensure that appropriate tables and entries were present
Analyze	An identical case to a local machine worked, now needs to work through code
Refine	Need to create a java program to manipulate the table data on the Ubuntu server
Prototype	Followed the same procedure as on the windows machine, wrote code which manipulated the local database table (ECE 356 + ECE 358 + ECE 454)
Test	Ensure that entries are present through MySQL commandline interface

The majority of design process uses knowledge from ECE 358 Computer Networks, ECE 356 Databases and ECE 454 Distributed Systems. At each cycle of the design iteration, observations have been made through rapid prototyping, and allowed for incremental changes to design which slowly inched towards a working solution.

3.2.3 Design Decisions and Alternatives Considered

UDP vs. TCP

For the transport layer the group considered two of the transport protocols, Transmission Control Protocol (TCP) and User Datagram Protocol (UDP). TCP offers a connection based service, while UDP offers a connectionless service. TCP offers greater reliability but lesser transfer time efficiency. UDP is less reliable however it is faster than TCP in terms of the transfer time. The transfer time was one of the criteria which would help the group meet the functional specs. [6] [7]

Wi-Fi

The group was also considering Wi-Fi as the medium of communication due to its widespread use and ease of scalability. Using any wired medium would be both expensive and impractical. Wi-Fi is cheap, but in rare places where it is not available, 3G/4G may be used as an alternative.

Centralized vs. Decentralized

Centralized solution offers most of the complexity at a single point and is simpler to design. In this case, a single server would run the software, make decisions and coordinate the garbage cans. A decentralized solution would have the clients (garbage cans) communicating between themselves and updating each other. Because of the generality of the topic, a good analogy that will be applied in the analysis of the decision would be based off similar ideas such as Link State and Distance Vector.

Message Format

An efficient message format means that maximum information is transferred from client to server and vice versa while keeping the messages short. Messages may include status updates, database queries and informational updates such as location.

3.2.4 Analysis

UDP vs. TCP

The group decided to go with UDP as the means of transport layer protocol. As learned in the ECE 358 Computer Networking, the UDP header size is 8 bytes, compared to TCP's header size of 20 bytes. This means that 12 bytes are wasted per transmission.

One of the functional requirements states that a garbage bins must transmit its status once per minute.

$$\# \text{ of transmissions per day} = 60 \text{ minutes} \times 24 \text{ hours} = 1440 \text{ transmissions/day}$$

$$\# \text{ bytes wasted per day} = 1440 \times 12 \text{ bytes} = 17280 \text{ bytes} = 17.3 \text{ kilobytes}$$

17.3 kilobytes of data would be completely wasted on metadata that the TCP provides for the services that we decided are not necessary for this project. This number gets multiplied proportionally as the number of garbage cans increases.

----- Server listening on TCP port 5001 TCP window size: 85.3 KByte (default) -----				----- Server listening on UDP port 5001 Receiving 1470 byte datagrams UDP buffer size: 208 KByte (default) -----			
[4]	local 198.50.229.155	port 5001	connected with	[3]	local 198.50.229.155	port 5001	connected with
[ID]	Interval	Transfer	Bandwidth	[ID]	Interval	Transfer	Bandwidth
[4]	0.0- 1.0 sec	69.9 KBytes	572 Kbits/sec	[3]	0.0- 1.0 sec	70.3 KBytes	576 Kbits/sec
[4]	1.0- 2.0 sec	68.1 KBytes	558 Kbits/sec	[3]	1.0- 2.0 sec	68.9 KBytes	564 Kbits/sec
[4]	2.0- 3.0 sec	68.4 KBytes	561 Kbits/sec	[3]	2.0- 3.0 sec	67.5 KBytes	553 Kbits/sec
[4]	3.0- 4.0 sec	65.3 KBytes	535 Kbits/sec	[3]	3.0- 4.0 sec	68.9 KBytes	564 Kbits/sec
[4]	4.0- 5.0 sec	65.6 KBytes	537 Kbits/sec	[3]	4.0- 5.0 sec	68.9 KBytes	564 Kbits/sec
[4]	5.0- 6.0 sec	68.1 KBytes	558 Kbits/sec	[3]	5.0- 6.0 sec	68.9 KBytes	564 Kbits/sec
[4]	6.0- 7.0 sec	68.4 KBytes	561 Kbits/sec	[3]	6.0- 7.0 sec	68.9 KBytes	564 Kbits/sec
[4]	7.0- 8.0 sec	68.1 KBytes	558 Kbits/sec	[3]	7.0- 8.0 sec	68.9 KBytes	564 Kbits/sec
[4]	8.0- 9.0 sec	68.4 KBytes	561 Kbits/sec	[3]	8.0- 9.0 sec	68.9 KBytes	564 Kbits/sec
[4]	9.0-10.0 sec	68.1 KBytes	558 Kbits/sec	[3]	9.0-10.0 sec	68.9 KBytes	564 Kbits/sec
[4]	10.0-11.0 sec	68.4 KBytes	561 Kbits/sec	[3]	10.0-11.0 sec	67.5 KBytes	553 Kbits/sec
[4]	11.0-12.0 sec	68.1 KBytes	558 Kbits/sec	[3]	11.0-12.0 sec	68.9 KBytes	564 Kbits/sec
[4]	12.0-13.0 sec	68.4 KBytes	561 Kbits/sec	[3]	12.0-13.0 sec	68.9 KBytes	564 Kbits/sec
[4]	13.0-14.0 sec	66.7 KBytes	546 Kbits/sec	[3]	0.0-13.4 sec	923 KBytes	564 Kbits/sec
[4]	14.0-15.0 sec	68.4 KBytes	561 Kbits/sec				
[4]	0.0-15.1 sec	1.00 MBytes	557 Kbits/sec				

Figure 7: TCP bandwidth (left) vs. UDP bandwidth (right)

This may not seem as much; however in the rare case where Wi-Fi is not available, this will help to reduce the cost of using mobile data networks such as 3G. From the cost point of view, this aids the group in meeting functional specification A). A tool called iPerf was used to quantitatively verify the varying transfer speeds as shown in the figure above.

`iperf -s -u -i 1` and `iperf -u -c fydp.minty.coffee -t 12 -i 1` commands were used for the UDP test and `iperf -s` and `iperf -c fydp.minty.coffee -t 12 -i 1` commands were used for the TCP test. The `-s` indicates the server side (receiver), `-c` indicates the client side (sender), `-u` indicates UDP, `-i` indicates the interval size for recording and finally `-t` indicates the amount of time the test is run for. As one can see on

right side of the figure, UDP bandwidth averages at around 564 Kbits/sec. For the same server and client, the TCP bandwidth averages at around 556 Kbits/sec. The team has concluded that UDP was more suited to specifications A) through E) since they're all time dependent. As learned in ECE 358, UDP is known to be used for real-time applications such as online gaming.

Another point of advantage for UDP is multicasting. A server may need to send out requests to multiple clients such as "return home" commands. Multicasting is not supported in TCP as TCP needs to establish end to end connection, and doing so between multiple points will be inefficient if it does succeed.

Centralized vs. Decentralized

In the case of a centralized solution, adding a new garbage can to the system would require only to add a new entry on the server side database. This is a $O(1)$ or a constant operation. With a decentralized solution, since all of the complexity is distributed among the clients (garbage cans), they would all need to be aware of a new addition to the system. Once a new garbage can enters the system, n garbage cans' database tables would need to be updated. Assuming each new garbage can inform its neighbors, the convergence time would be logarithmic as shown in figure below. However the amount of traffic generated would be much greater. The amount of traffic generated by the clients would get worse and worse as the number of them increases. This trend can be observed in Figure 8.

Additionally, implementing a distributed system would be a more difficult task. As learned in ECE 454 Distributed Systems, independent systems would need to be synchronized such that the time stamped actions are still valid. Out of order queries may result in stale data being sent over to the neighboring clients. This can cause data corruption.

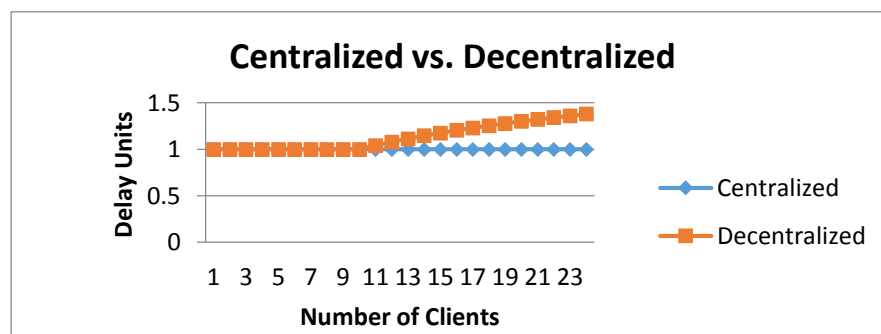


Figure 8: Theoretical Delay Measure

The group concluded that the engineering cost that is required for a decentralized solution does not justify its implementation. A good technical analogy for centralized vs. decentralized would be the two routing algorithms: Link state and distance vector. As learned in ECE 358, Link state uses Dijkstra's [8] algorithm to calculate the shortest path. Distance vector algorithm uses Bellman-Ford algorithm to calculate its distances, which inherently involves its neighbors which is demonstrated in the equation: $Dx(y) \leftarrow \min_v \{c(x,v) + Dv(y)\}$. (Note that it is a recurring relation). The reason for this analogy is that both of these routing algorithms share some of the common pros and cons observed in practice of centralized vs. decentralized systems. Link state for example, is capable of broadcasting information to all nodes. It is impractical in larger networks as sending information to all nodes simultaneously causes major congestion

in the network. However, by doing so, it is the fastest in updating every node. Distance vector on the other hand, uses a recurring relation, which passes information from neighbor to neighbor. While it is easier on congestion, the time it takes for the messages to propagate increases as the number of nodes increases. This means increased delay. As discussed in distributed systems, there must also be a mechanism for determining if any nodes updated while someone else is propagating their updates. Keeping the delay down helps the group meet deadlines for larger systems as required by functional specs A) through E).

Message Format

Received messages will modify the database tables so that the tables could be used for the allocation algorithm. The message format needs to be easily comprehended because the server will then create a database query in the form of:

String constructing = "INSERT INTO TableName(Column1, Column2 ...) " + "VALUES(Value1, Value2 ...)";

Hence columns and values must be easily extractable from the single string that the server receives. In order to accommodate such a constraint, an appropriate message format is needed. The reason that string parsing is done on the server side is because it reduces the client side complexity. As seen in Figure 5 below, bin entity has a number of attributes. Attributes such as last_updated are not provided by the client, rather they are time stamped by the server upon receiving the message. As a group, we determined that each garbage bin will need to send the following attributes to the server: ID#, battery life, remaining volume capacity, coordinates.

Example string that will be received from the client may look like the following:

01, 0.98, 0.70, 55.00.00.0, 55.00.00.0

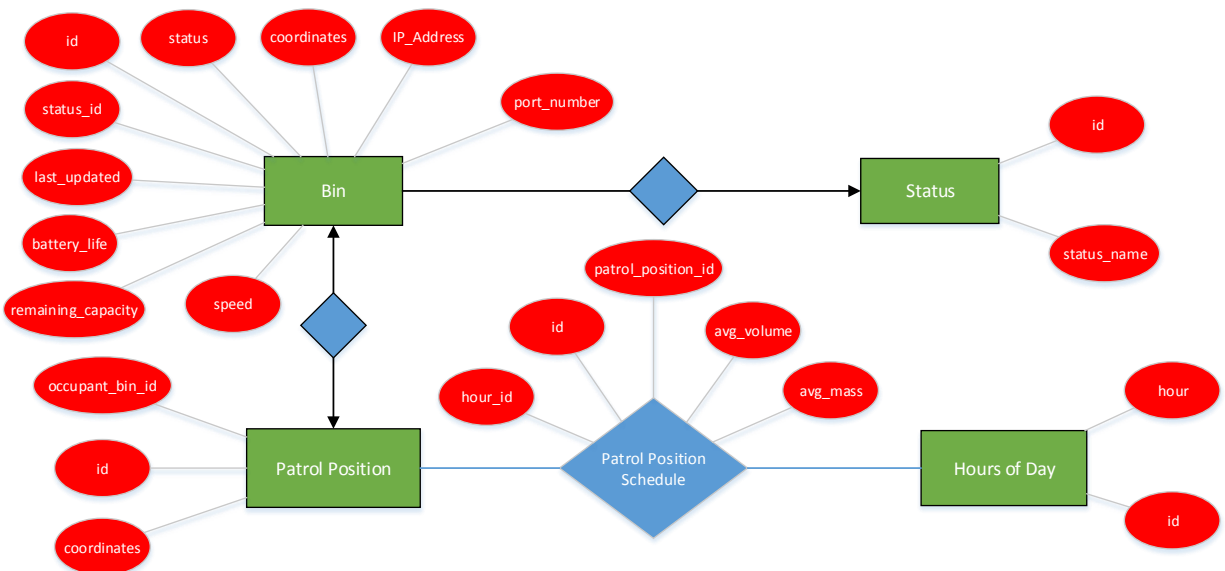


Figure 9: ER Diagram of Database

The values will need a separator, either a space or a comma would work. The 4 attributes effectively identify all the data that will be required by the database table, such that no bandwidth is wasted.

3.3 Load Sensor Subsystem

The load sensor is a subsystem designed to detect when a container is full and how full it is at any given time.

3.3.0.1 Hardware

The load sensor hardware will consist of two ultrasound sensors spaced on the opposite ends of the lid of the container. The sensors will interface with the microcontroller using the standard built in GPIO pins. For this design, the HC-SR04 Ultrasonic Sensor was picked due to its relatively low cost, easy availability and decent precision.

The HC-SR04 comes with both an ultrasound emitter and a receiver. Once the trigger pin is set to high, it emits ultrasound pulses and the echo goes high while waiting for a response. By measuring how long the echo stays high and dividing by the speed of sound, the distance the wave travelled can be measured.

Given that the speed of sound varies due to air temperature, a temperature sensor was added to help estimate a more accurate speed of sound. The temperature sensor used is the TMP36GT97Z. It was picked because of its decent versatility and the fact that it does not need external calibration for decent precision. This sensor's output voltage linearly increases with temperature. To measure the output voltage, it needs to be connected to a microcontroller's Analog to Digital Converter.

3.3.0.2 Software

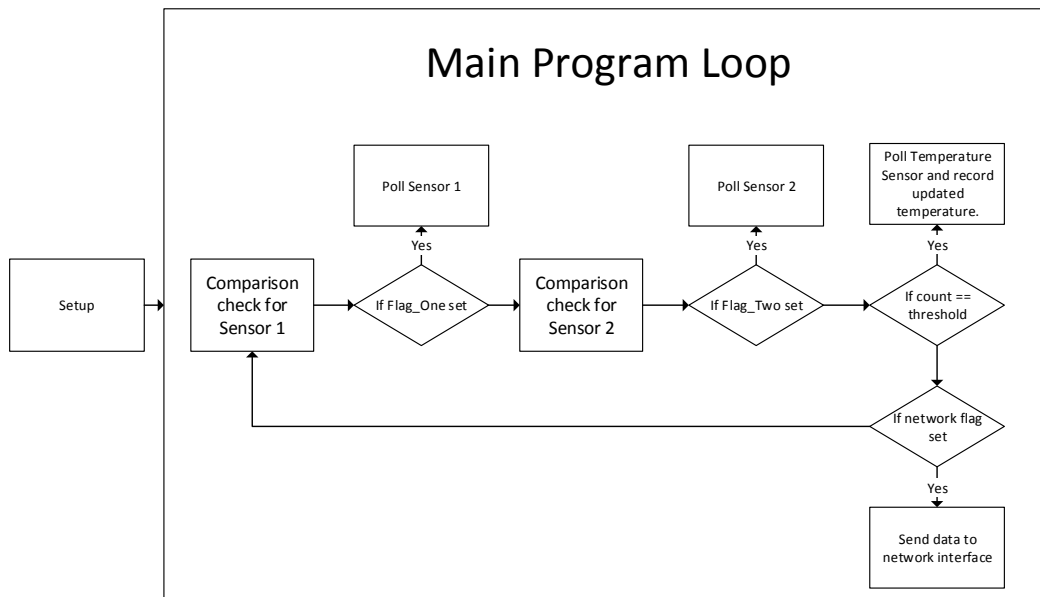


Figure 10 The Main Program Loop for the Load Sensor Software

The software running atop the microcontroller that interfaces with the ultrasound sensors is a simple timer interrupt based routine. Figure 10 illustrates the main program loop. The flags and the count variable are all set by a timer. A timer will fire every T seconds (where T is a pre-set polling rate) and the

interrupt service routine that got started will set a global flag corresponding to a given ultrasound sensor to true. The next time the ISR runs, the other ultrasound's flag will be set to true.

The ISR also increments a counter and once this counter has reached a certain threshold, the main loop polls the temperature sensor to get a more updated value for temperature that can be used to calculate the speed of sound as per the equation: $v = 331 \frac{m}{s} + \left(\frac{0.6 \frac{m}{s}}{C} \right) * Temperature$ [9]

The software cannot and should not use both sensors at the same time because their respective sound pulses will interfere with one another and cause discrepancies in the measured value.

Back in the main loop, that flag will be checked and if it is set to true, the microcontroller will use the ultrasound sensor to get a measurement.

Getting a measurement from the ultrasound sensors entails setting the trigger pin of HC-SR04 to high and then counting how long the echo pin is high for. Once a count is found, the last known speed of sound can be used to figure out the distance in our unit of choice.

While polling the sensor, the timer interrupts are disabled because an interrupt at this time is likely going to throw off the distance estimate. The interrupts are re-enabled once the value is recorded.

While not gathering new data, the software does comparisons on which data it should keep.

The software keeps track of the past 3 readings for each sensor to mitigate the amount of erroneous values. Until a total of 3 readings are done, the software does nothing and simply waits for the ISR to fire. Once it has collected three values, it compares the second value received to the first and the third. If the value differs by $\pm 40\%$, it is discarded and the third value is moved up and becomes the new second.

If the value does not differ then the first entry is discarded, the second and third move up to become the first and second respectively. This comparison is done for all the sensors in the system.

The values (data1, data2 and data3) are all stored in individual variables and a "last good value" variable exists for both sensors. These variables hold the value that the microcontroller will send to the Network Interface when the network interface timer interrupt (a longer timer) is triggered. When the interface timer is triggered, another flag is set and the main loop upon checking this flag will send an average of the "last good values" to the network interface.

3.3.0.3 Project Specifications Satisfied

The specifications that need to be satisfied by the load sensor are:

- There must be a latency of under 5 seconds when measuring a load.
- The embedded software must be able to measure and report garbage load measurements within $\pm 10\%$ accuracy of a manually conducted garbage load measurement.
- Sensor readings that differ by $\pm 40\%$ from previous and next readings should be discarded as outlier values.
- The system shall last at least 6 hours on a single battery charge.

- The garbage bin must broadcast a request for garbage collection when it is within 90% to 100% filled to capacity.

This design meets these specifications to an acceptable degree. Assuming a worst case sound speed of 330m/s, and an absolute max distance of 4m (the sensor's limit), the ultrasound pulse will take at most a time of $4m * \frac{2}{330m/s} = 0.024 s$.

If polling is done every half second or so (polling both sensors will then take 1 second), then assuming no erroneous values, a value for the capacity of the container will take around $(1 + 2 * 0.024) * 3 = 3.144$ seconds.

If a single erroneous value is found, it will still take under 5 seconds, but if the application of the system ends up showcasing more frequent erroneous values, the polling rate can be increased up to a maximum of 60ms (specified in the sensor spec). The latency should never reach 5 seconds should that polling rate be used.

Note that the previous calculation assumed that there wasn't any data already in the system. In general, there will be 2 previous entries in the system already and only the third will count towards latency. This will divide the average measurement time by a third.

The ultrasound sensors used for this design, advertise an accuracy of up to 2mm. Testing has verified that this is a reasonable claim. This degree of accuracy is most certainly within $\pm 10\%$ accuracy of a manually conducted measurement. Unfortunately, there are cases where this accuracy does not hold true. Uneven objects might throw off the measurements a bit and very porous materials might deceive the sensors as well, so reaching that $\pm 10\%$ might not be possible for all cases, but in the average case (mostly flat, mostly hard surface), it should be attainable.

The software design discussed in the detailed description clearly handles the outlier requirements. Entries that differ by 40% are discarded and not used for capacity measurements.

The specification for battery should be achievable. Each ultrasound sensor consumes at most 2mA of current when idle and at most 15mA when active. If we assume the sensors will be polled every second (every half second one of them is polled), then we get $15mA * 60 * \frac{1}{60} = 15mAH$. The temperature sensor consumes less than 50 μA of current when active and 0.5 μA when "shutdown". Even if it is left on the entire time, it will consume at most $50 * 10^{-3} mA * 60 = 3mAH$ of current. As long as the battery is capable of $6 * (3 + 15) = 108 mAH$, the load sensor system should be fine (note that this does not include the cost of the microcontroller or the other subsystems).

3.3.1 Design Iterations

On the hardware front, the initial design consisted of a single ultrasound sensor. An additional sensor was then added to gain larger coverage. The thermal sensor was then added in to try and improve the accuracy of the measurement.

The software has gone through several iterations before arriving at the current design. Initially, it was thought that simple, constantly polling system would suffice. However, this wouldn't allow control over the polling frequency and would thus be detrimental towards optimizing battery life and general system use. Not to mention, other important tasks that need to be done (such as examining the data and sending it to the network interface) would be hard to fit in there.

When it was decided that a timer would control the rate of polling, it was initially suggested that the ultrasound polling would be done in the interrupt service routine itself. That is to say, the system would always be examining its values (now held in a fixed sized array) and whenever the ISR fired, it would simply add to this array. Discarded values would be nulled out and eventually overwritten.

This design was abandoned partly because of the additional complexity of keeping a constantly shifting array but mostly because of the large size of the ISR. ECE courses like ECE 222 and ECE 224 strongly advocate for small interrupt routines because large ones can cause many problems. A large routine could for example get interrupted frequently by other ISRs (and this was not ideal for a time dependent polling routine like this) and the memory management for larger routines also places greater stress on the system.

To combat this, it was decided that the timer based ISR would simply set a flag that would be checked by the main loop at its leisure. This of course meant that the polling frequency would not be as precise as set, but given the relatively small amount of time spent in other parts of the program, it would be precise enough to not matter.

Similar reasoning was applied to the network interrupts. Initially, the network ISR would take the data and write it out to the network interface, but this was changed to it just setting a flag and the main loop instead just checking that flag and taking action when it had time.

These changes diminished the responsiveness of the system, but given that the system does not need to react rapidly to external changes, this was a worthwhile trade-off for greater simplicity, less variability and hopefully greater stability (less prone to timing problems).

3.3.2 Alternatives Considered

There were several alternatives considered for the sensor choice. These alternatives consisted of a camera, an infrared grid or a combination of infrared and ultrasound.

The camera would have been a wide-angled infrared camera capable of taking pictures in the dark. Alternatively, the design would consist of two or more smaller cameras and the images would be stitched together. A visible light camera with a flash was also considered, but discarded shortly after because of the power requirements for the flash (bad battery life). On the sides of the container, there would be markers indicating distance. A picture of the container's contents would include these markers and the image could then be analyzed using image processing to determine how far from the top the items were (i.e. how full the container was).

The infrared grid would constitute of several infrared emitters arranged in a grid like fashion on two sides of the container. There would be vertical levels that when blocked would indicate a certain capacity was

for sure reached and horizontal levels that would help give a more accurate picture. The entire vertical line could for example be covered with a thin cardboard box but there would still be plenty of space in the container. The horizontal lines would alleviate this problem.

The final alternative would constitute a single horizontal line of infrared emitters that would help nullify the ultrasound sensor's problems near the edges of the container (ultrasound cannot work near the edges because of interference introduced by the container wall). If a user placed a large piece of cardboard on the side for example, the emitters would be able to catch that while the ultrasound alone might not be able to do so.

In order to choose the correct option, weights were assigned to important criteria and then each option was ranked accordingly in a decision matrix. The weights were assigned as follows:

Table 7: Assigned Weights

Criteria	Simplicity	Accuracy	Consistency	Resource Savings	Monetary Savings
Weight	30%	20%	20%	20%	10%

Simplicity was the most important due to a limited time frame. Complexity also increases the likelihood of something breaking and that is not ideal for an outdoor, generally out of reach embedded system. Accuracy was assigned an important weight of 20% because of the $\pm 10\%$ accuracy spec. Consistency refers to how often the most accurate measure can be had. Resource savings was also given a pretty high weight because CPU time and battery resources are limited and there is a requirement for battery time that needs to be met. Monetary savings are important but not as important as the other requirements, so it was assigned a lower weight. Table 8 shows the alternatives ranked by how well they met the specified criteria.

Table 8: Ranked Options

Camera Option	Simplicity	Accuracy	Consistency	Resource Savings	Monetary Savings	Total
Infrared Camera	$\frac{5}{10} * 30 = 15$	$\frac{7}{10} * 20 = 14$	$\frac{9}{10} * 20 = 18$	$\frac{3}{10} * 20 = 6$	$\frac{2}{10} * 10 = 2$	55/100
Infrared Grid	$\frac{6}{10} * 30 = 21$	$\frac{6}{10} * 20 = 12$	$\frac{8}{10} * 20 = 16$	$\frac{6}{10} * 20 = 12$	$\frac{5}{10} * 10 = 5$	66/100
Ultrasound	$\frac{8}{10} * 30 = 24$	$\frac{8}{10} * 20 = 16$	$\frac{7}{10} * 20 = 14$	$\frac{9}{10} * 20 = 18$	$\frac{8}{10} * 10 = 8$	80/100
Ultrasound + Infrared	$\frac{7}{10} * 30 = 21$	$\frac{8}{10} * 20 = 16$	$\frac{8}{10} * 20 = 16$	$\frac{7}{10} * 20 = 14$	$\frac{6}{10} * 10 = 6$	73/100

The infrared camera lost points for simplicity because interfacing with a camera is difficult for a microcontroller especially if the camera is of a resolution high enough to provide the images required for processing. The image processing is also very expensive CPU cycle/battery wise and thus there is a significant loss on resource savings. Infrared cameras are also very expensive especially if a wide-angled one is used. The accuracy is pretty decent assuming a well implemented algorithm. It loses points here because it uses fixed markers but it would be able to meet the $\pm 10\%$ requirement.

The infrared grid is complex because it involves interfacing with a number of infrared sensors and managing them without running out of pins on the microcontroller requires additional hardware. This is also why it gets a low score for both resource and monetary savings. Accuracy is low because there are only a certain number of discrete vertical levels that it can measure to. Consistency is quite high, but not perfect because certain materials like glass and mirrors can throw off the readings a fair bit.

Ultrasound is simple to use (two ultrasound sensors and a thermal sensor), quite accurate (to a few mm in the best case), very resource friendly (sensors draw little current when not in use and the data doesn't require much processing. The materials can also be found for reasonable prices so monetary cost isn't too high either. The one slightly less great metric is consistency due to uneven/porous materials throwing off the readings, but even that is not too bad.

Ultrasound plus infrared has most of the advantages of ultrasound with a slightly higher consistency (infrared helps deal with items solely on the walls and prevents overflowing) but at a slight cost of simplicity, resource savings, and monetary savings.

As is evident from Table 8, ultrasound by itself was the best option for the sensor.

3.3.3 Analysis

As shown in the Alternative Implementation section, the ultrasound sensor configuration is the best option for meeting the requirements. As demonstrated in the Functional Specifications section, all of the specifications that are relevant to the Load Sensor are met by this design.

Naturally, practical testing will have to be done when the prototype is complete to verify this claim. In the unlikely case that the measurements are not consistent enough due to ultrasound's weakness around the sides, the design can be modified to include a layer of infrared emitters without too much additional work though this will likely not be required. This is especially true because a container with a large amount of items on the side and nothing in the middle can and likely is considered by most people to be not full.

In terms of software, the greatest risk comes from timing problems introduced by the use of timers and not being able to service them immediately. This shouldn't pose much of a problem though if the code is written well. The system does not need to react especially fast to external changes since it is in a mostly stable environment (items are generally not thrown away very rapidly and when they are, it is only for a short amount of time).

3.4 Battery Sensor Subsystem

3.4.1 Detailed Description

The Battery Sensor will require a separate connection to the main battery and a reference voltage using a 5V voltage regulator such as the LM7805C [10]. It consists of an interrupt using a digital comparator and an analog voltage input after a voltage divider into the analog input pin of the microcontroller. Using the microcontroller's onboard Analog to Digital Converter, we can accurately read the current voltage being outputted by the battery.

The circuit diagram of the sensor is as follows:

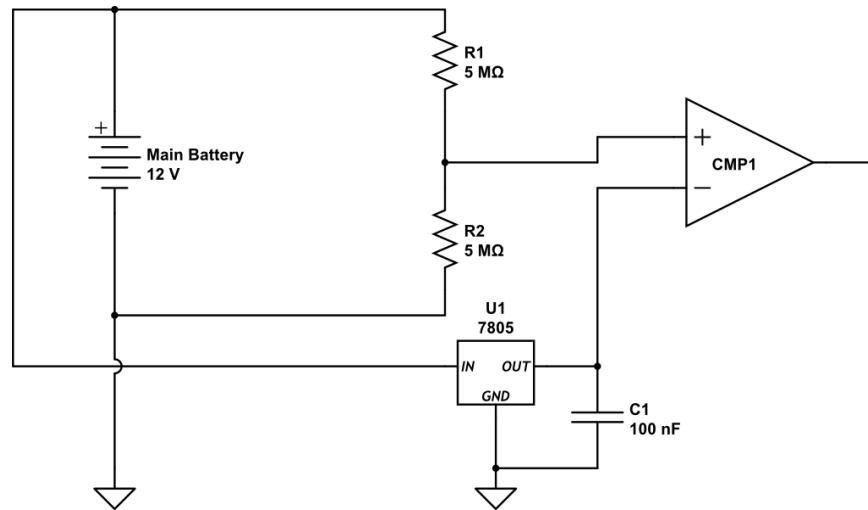


Figure 11: The Circuit Diagram for the Battery Sensor Subsystem

The comparator's positive input will also be connected to the microcontroller's analog input pin to enable the microcontroller to read the divided voltage level from the battery. The output of the comparator will be connected to a digital input pin of the microcontroller. A 1 (5V) output from the comparator indicates that the battery is above 10 volts and is able to provide adequate power to the microcontroller and the rest of the subsystems. When the comparator output drops to a 0 (0V), it indicates that the battery voltage is below 10 volts and will be considered to be low power.

The microcontroller code could set up an interrupt from the digital pin of which the comparator will be connected to and have an ISR to enable notification and appropriate responses when the battery level drops below 10 volts. This also has the added benefit of notifying the microcontroller when the battery level has been sufficiently recharged (above 10 volts).

The simulation result of the circuit is shown below:

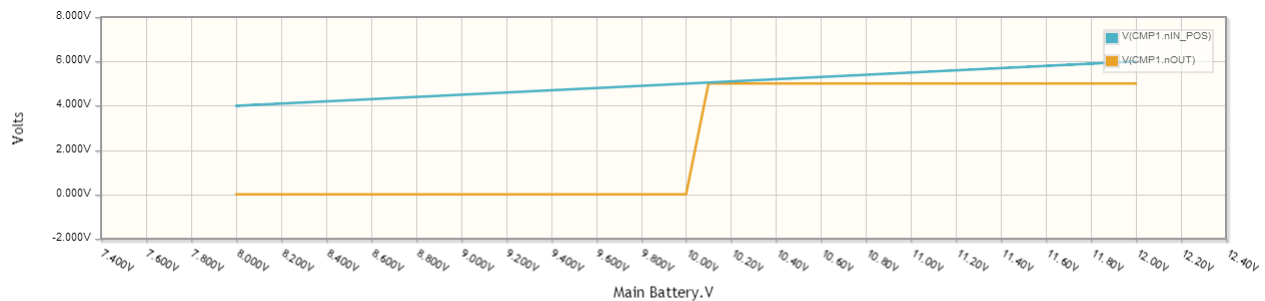


Figure 12: The Simulation Result of the Circuit in Figure 11

3.4.2 Project Specifications Satisfied

The specifications related to the battery sensor are:

- The garbage bin must be able to measure its remaining battery level. The accuracy of these battery level measurements should be within $\pm 10\%$ of the measurements conducted using a commercial tester.
- The garbage bin must broadcast a request for service if its battery level runs below 20%.

This design meets these specifications to an acceptable degree.

The voltage divider will draw on average 1.2 μA of power when the microcontroller is not sampling the voltage level. The LM7805C voltage regulator's datasheet shows that the regulator's Quiescent Current (current it draws when it is not driving a load) is 6mA at 25 degrees Celsius and 6.5 mA when it is over temperature. Therefore, the entire circuit should draw approximately 6.5 mA of current when idling.

Assuming the battery used to power the unit is at a minimum of 10 ampere-hours. It will power this circuit on average for $10/0.0065 = 1538$ hours. Since the microcontroller and other subsystems will be requiring substantially more power on average, this circuit's contribution to the power consumption is negligible.

3.4.3 Design Alternatives Considered

There are several other methods of building a circuit that can accurately detect the battery's output voltage such as through the use of operational amplifiers and transistors. However this design was deemed to be the most simplistic, cheap and reliable method since it only relies on components that are already a part of other subsystems.

4. Discussion and Project Timeline

4.1 Discussion on Design

4.1.1 Scheduling to Date

As of the writing of this report, group members have put in a variable number of work hours into the design and overall requirements. Specifically, the table below demonstrated the hours to date logged by each group member. Given the recommendation of approximately 10 hours a week per group member, it is likely that as the term continues, group member include some more time per week as to reach the 10 hour a week average and ultimately complete the design in timely order.

Table 9: Group Member Logged Hours to Date

Group Member	Hours Logged to Date
	75
	120
(names removed)	79
	68
	42

A reason for the varying hours in each member's log is the fact that some members are taking on more significant design components of the overall project. Specifically, member XXXXX contributed dozens of hours of his own time toward the project in the month of June researching with regards to the

allocation design aspect of the project. Additionally, the differing class and midterm schedules in June affected the abilities of some members to log a sufficient amount of time towards the project.

4.1.2 Upper Year Knowledge Incorporated

During the design stages of the project, several upper and lower year ECE courses greatly aided in the project progression. For the garbage bin allocation subsystem, knowledge from ECE 457A regarding stochastic search algorithms such as simulated annealing was incorporated into the initial prototype algorithms. Likewise, knowledge regarding probability and statistics from ECE 316 was central in the development of a cost function for the garbage bin allocation subsystem. For the network communication subsystem, basic theory of client and server communication learned from ECE 358 such as the nuances between the UDP and TCP Transport Layer protocols helped to lay out the foundation for the initial server communication prototype. As for the battery sensor subsystem, knowledge regarding linear circuit analysis (e.g., voltage dividers) from ECE 140, ECE 240, and ECE 242 was necessary for the design of the physical circuit. Finally, knowledge regarding hardware interfacing and I/O (e.g., polling vs. interrupts) from ECE 222, ECE 224, and ECE 455 was necessary for determining how meaningful data should be read from the battery sensor and load sensor subsystems into the relevant microcontrollers.

4.1.3 Critical Analysis

In general, the essential and non-essential project specifications known to be satisfied by the subsystem designs defined in this document are explicitly outlined and explained in sections 3.1.4.2, 3.2, 3.3.0.3, and 3.4.2 for the garbage bin allocation subsystem, the network communications subsystem, the load sensor subsystem, and the battery sensor subsystem respectively. From this list of specifications, it can be asserted that the subsystem designs proposed in this report satisfy most of the essential specifications defined in Section 2, as well as some of the non-essential specifications. However, there are three essential specifications that have not been included as part of any of the subsystem designs analyzed in this report (i.e., functional specifications 4 and 8 and non-function specifications 2 in Section 2). In this case, functional specification 4 is expected to be satisfied through a simple linear circuit connecting an LED to a microprocessor pin that turns on whenever the garbage bin disconnects from the server for a period of time. Meanwhile, functional specification 8 is expected to be satisfied as a corner case in the network communications program running on the garbage bin. In neither case are the subsystems responsible for each of these specifications particularly challenging enough to warrant analysis in this document due to time and space constraints. Finally, non-functional specification 2 is expected to be satisfied by the prototype and final implementation of this project, as this document in particular does not explicitly analyze the physical placement and organization of circuits and sensors in the garbage bin.

While all the essential specifications are expected to be satisfied by a prototype resulting from the design provided in this report, it is likely that the project itself would benefit from further iteration and refinement. For instance, further iteration and refinement of the garbage bin allocation subsystem may yield a more time efficient algorithm for garbage bin allocation. Additionally, further iteration and refinement in this project may allow the design of other subsystems required for the satisfaction of non-essential specifications in this project to be fleshed out. A good example of this is the autonomous navigation subsystem, which was not considered in this document due to timing constraints this term.

From a novelty perspective, while smart garbage bins do exist in practice, it is rare to find a system of garbage bins that can be collected on demand and dynamically reallocated based on usage statistics. The novelty of this design lies in the fact that the system is able to learn over time which patrol positions produce more garbage at which hours of the day, which allows the garbage bin allocation policy to effectively adapt to changes in usage patterns throughout the day. Additionally, the design is intended to bring humanity one step closer towards completely automating the seemingly-mundane job of garbage collection, especially if autonomous movement and navigation of the garbage bin becomes a feasible target later in the project.

4.1.4 Potential Hazards

The only potential hazards that are anticipated for this project concern the battery sensor subsystem. In particular, the power source for each garbage bin will most likely be a high capacity lithium ion battery of some form. In order to measure the remaining power level of the garbage bin, the battery sensor subsystem which itself is a custom circuit will presumably need to be connected to the battery in question. Hence, special precautions will need to be taken to ensure that the battery sensor subsystem operates safely, which may involve further design iterations to the battery sensor in the near future.

4.2 Project Timeline

There are a number of tasks required to be completed through the duration of this project. The following 11 tasks were determined to be necessary for the design and analysis of the overall project:

1. Ordering Parts and Delivery
2. Sensor Detector Design
3. Allocation Algorithm Design
4. Communication Setup Design
5. Navigation Algorithm Design
6. Battery Detector Design
7. Garbage Bin Hardware Design
8. Integration of Design Components
9. Testing and Final Prototype Demo
10. Prepare Final Report
11. Prepare Design Symposium Presentation

First the tasks were ordered in such a way that those tasks required most or which others are dependant are schedules first. Noting that there is an expectation of having a 50-75% complete prototype by the end of the 4A semester, it is very important that these tasks be planned out and given time so that this specification can be met. To do this, it is required that a heavy portion of these design component tasks be scheduled for completion by the end of the 4A semester. Finally, those tasks which are obviously related to deadlines given in the 4B semester are ranked lower in priority on the list (specifically the final report and symposium presentation).

Given the above notes, the Gantt chart below was generated. Luckily since a very large portion of this design project is in the realm of software (i.e. not physical), the group is able to allocate many of the listed

tasks to be scheduled in the remainder of the 4A semester such that the 50-75% requirement can be met. Additionally, given that these software and network components of the design project are relatively independent of one another, they can be scheduled simultaneously as to not spread out the weeks over which they are completed. Additionally, if it occurs that, at the time of presenting the design prototype at the end of the 4A semester, there are some design faults, then some time could be allocated during the co-op term between 4A and 4B as required. These times during the co-op semester and during final exams are in grey in the Gantt chart to signify the fact that there is an expectation of minimal design work to be done during these times.

During the final 12 weeks on the Gantt chart, many of the tasks are hardware heavy and tasks which bring together the several subcomponents. Further, the final three tasks are scheduled simultaneously as there is the expectation that they will not necessarily interfere with one another. The testing test is given ample time so that, should something go wrong, the necessary fixes can take place. Finally, the final report and design symposium presentations are both scheduled to be completed in mid-March each starting 4 weeks before that time. It is the expectation that 4 weeks is sufficient (but necessary) to complete these report and in order to start said tasks, it is required that integration of all subcomponents be completed first.

Note that each division along the horizontal is 4 weeks

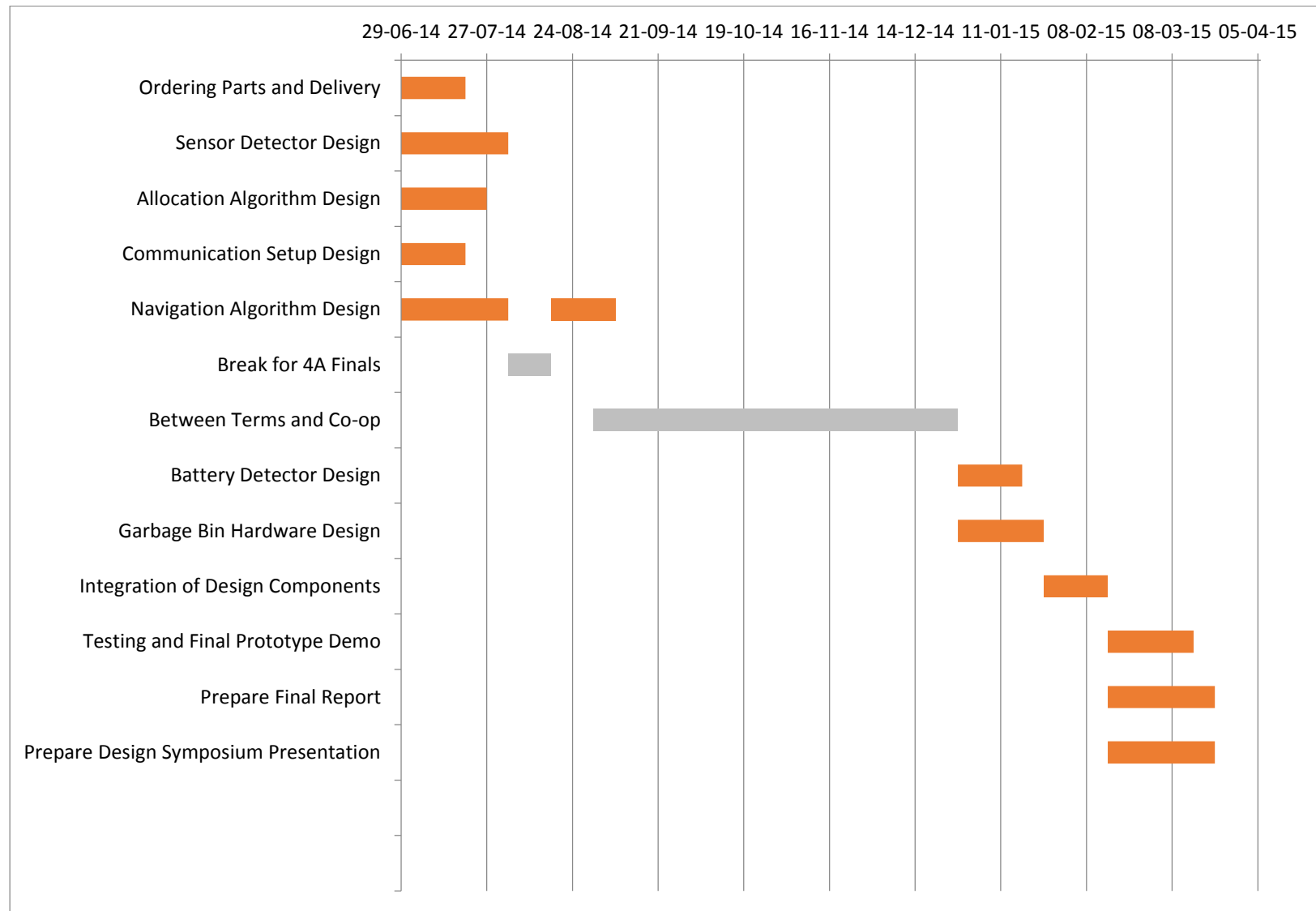


Figure 13: Project Management Gantt Chart

References

- [1] F. Council, "Litter facts: why do people litter?," [Online]. Available: http://www.falkirk.gov.uk/services/corporate_neighbourhood/estates_management/litterzone/litter_strategy/litter_facts/why_do_people_litter.aspx. [Accessed 24 May 2014].
- [2] L. I. W. TOO!, "Why do people litter?," [Online]. Available: <http://www.litteringiswrongtoo.org/faq#people>. [Accessed 24 May 2014].
- [3] M. Badreldin, "A Metaheuristic Approach to Multi-Robot Task Allocation," Cairo, 2013.
- [4] M. Badreldin, A. Hussein and A. Khamis, "A Comparative Study between Optimization and Market-Based Approaches to Multi-Robot Task Allocation," Hindawi Publishing Corporation, Suez, 2013.
- [5] D. e. a. Davenda, Traveling Salesman Problem, Theory and Applications, Rijeka: InTech, 2010.
- [6] J. Postel, "RFC 768: User Datagram Protocol," Internet Engineering Task Force, 28 August 1980. [Online]. Available: <http://tools.ietf.org/html/rfc768>. [Accessed 2 July 2014].
- [7] Information Sciences Institute, "RFC 793: Transmission Control Protocol," Internet Engineering Task Force, September 1981. [Online]. Available: <http://tools.ietf.org/html/rfc793>. [Accessed 2 July 2014].
- [8] G. Pandurangan, V. S. A. Kumar, M. Gu, Y. Liu and Y. Li, Wireless Algorithms, Systems, and Applications, Beijing: Springer Science & Business Media, 2010.
- [9] The Physics Classroom, "The Physics Classroom," [Online]. Available: <http://www.physicsclassroom.com/class/sound/Lesson-2/The-Speed-of-Sound>. [Accessed 30 06 2014].
- [10] National Semiconductor, "LM340/LM78MXX Series 3-Terminal Positive Regulators Datasheet," February 2000. [Online]. Available: <http://www.jameco.com/Jameco/Products/ProdDS/51262.pdf>. [Accessed 28 June 2014].